

SmartFlow: Adaptive Traffic Signal Control with Reinforcement Learning

Final project report | Code: https://github.com/yq688m3JT/final_report

Problem and Motivation

Urban traffic signals are a sequential decision problem: each green phase relieves one movement while delaying others, and the best action depends on future arrivals as much as the current queue. The slide presentation framed this as SmartFlow, a tabular reinforcement-learning controller for an intersection with North-South and East-West traffic. This report first validates the slide experiment, then extends it to a small network that better resembles real city operation.

Slide Experiment: Single-Intersection MDP

The original model is a discrete-time Markov decision process. The state is $(Q_{NS}, Q_{EW}, \text{phase})$, where each queue is bucketed into Low, Medium, or High and the phase is either NS-green or EW-green. The action is stay or switch. The reward is $R = -(Q_{NS} + Q_{EW}) - C I(\text{switch})$, so the controller is penalized for total waiting vehicles and unnecessary phase changes. This matches the slide content and the original code preserved in `legacy_efficiency_experiment.py`.

The validation run confirms the slide's qualitative result: adaptive RL improves over fixed-time control, while value iteration is the full-information ceiling and the greedy heuristic is unusually

Approach	Average reward	Interpretation
Optimal value iteration	-378.92	Best full-state planning baseline.
Greedy heuristic	-394.84	Strong local oracle for a simple intersection.
Monte Carlo	-430.46	Learned adaptive policy; higher variance.
Q-learning	-443.72	Learned adaptive policy; beats static.
SARSA	-453.93	On-policy learner close to static.
Static fixed cycle	-456.24	Traditional non-adaptive baseline.

Network Extension: Real-City Traffic-Light System

The extension changes the project from one isolated intersection into a 3x3 city grid. Vehicles enter from the grid boundary, follow through and turning routes, queue at intersections, and exit at the opposite side. Each intersection has a capacity limit, so a green light can fail to discharge vehicles when the downstream intersection is full. This spillback behavior is the main reason real traffic networks are difficult: serving the longest local queue is not always good if it pushes cars into a blocked downstream link.

The extended RL controller uses a shared tabular Q-function across intersections. Its local state includes NS demand, EW demand, current phase, and a downstream-congestion bucket. The action remains stay or switch. The training reward penalizes total network queue, blocked movements, and switching. This keeps the model close to the original slide logic while adding the missing network effect: coordination through downstream awareness.

Experimental Design

I compared four controllers over 80 evaluation episodes after training: a traditional fixed-time cycle, a greedy longest-queue rule, a max-pressure heuristic, and the trained tabular RL controller. Demand varies during each episode with a rush-hour pulse and a lane bias, so the system is non-stationary. The primary metrics are average network queue, completed-trip delay, throughput, and number of phase switches.

Controller	Avg. queue	Avg. delay	Throughput	Switches
The key claim is supported against the traditional fixed-time algorithm: RL reduces average network				
Static fixed-time	42.96	15.45	692.80	180.00
Greedy longest-queue	21.17	8.12	707.25	464.10
Max-pressure heuristic	21.90	8.36	706.74	506.53
Tabular RL network	21.70	8.29	706.00	540.00

Why RL Helps in the Extension

The fixed-time controller ignores demand, so it keeps granting green time to low-demand movements during the rush pulse. The RL controller learns from repeated simulated episodes that switching should be conditioned on both local demand and the downstream state. This matters because a local queue can be misleading: clearing it into a saturated downstream intersection only moves congestion rather than reducing it. The learned policy is still tabular and interpretable, but it has enough state information to react to network pressure instead of only following a clock.

Limitations

The simulator is intentionally compact so the entire project can be reproduced quickly. It does not model pedestrian phases, yellow/all-red clearance, turning-lane geometry, emergency vehicles, or calibrated real detector data. The tabular state buckets also create a discretization gap: the agent cannot distinguish all queue lengths or platoon structures. These limitations explain why sophisticated heuristics with real-time queue access remain competitive in the small grid.

Conclusion

The original presentation result is real: the project code implements an MDP traffic-signal controller and shows that learned adaptive control can outperform a fixed-cycle baseline. The network extension strengthens the project by testing the same idea in a more realistic setting with interacting intersections, time-varying demand, route movement, and spillback. The learned RL controller performs much better than traditional fixed-time signal timing and approaches the performance of stronger state-aware heuristics. Future work should replace queue buckets with continuous observations and use deep RL or multi-agent actor-critic methods so the controller can coordinate across larger city networks without hand-designed discretization.

Network extension: delay by controller

